Signal Processing Project

AUDIO WATERMARKING

Authors

Luparu Ioan-Teodor 333 **Savin Radu-Andrei 352**

Abstract

This project acts as an introduction to the audio watermarking technique. It presents fundamental concepts of audio watermarking and goes into detail about various algorithms used for hiding information inside audio signals. The aim of the project is to showcase the strengths and weaknesses of these algorithms by pitting them against destructive signal processing operations and analysing which ones are more resilient and why.

Contents

1 Introduction **4**
1.1 Hiding Information 4
1.2 Audio Watermarking 5
1.3 Design Requirements 5
1.4 Types of Watermarking Methods 7

2 Implementation of Algorithms **9**
2.1 Audio Watermarking Algorithms 9
2.1.1 Time Domain Methods 9
2.1.2 Transform Domain Methods 11
2.2 Reed Solomon Codes 14

3 Experiments and Results **16**
3.1 Setup 16
3.2 Results 18

4 Conclusions **22**

5 Possible Improvements and Future Work **24**

Bibliography **26**

Chapter 1

Introduction

1.1 Hiding Information

The rise of the internet has greatly improved media accessibility. People can use the web to easily find, download and share images, music, videos and other forms of digital content. Enforcement of copyright and ownership over such content has become increasingly necessary, but also much more difficult. Various sophisticated techniques have been developed to help content creators protect their work from unauthorized redistribution and piracy and some of them involve the use of steganography.

Steganography is the practice of embedding data within a message or another piece of information such that an unsuspecting individual cannot detect its presence. Any type of digital content can be hidden within another file that is oftentimes of a different format altogether from the original. Compared to Cryptography, where the focus is on obscuring private data, steganography instead bases itself on concealing the existence of secret messages.

1.2 Audio Watermarking

Audio is one of the easiest forms of content to share and distribute online, making it a prime target for unauthorized use. To illegally copy an audio file is as easy as recording a playback of it. Therefore, techniques that aim to protect audio content should utilize the audio signal itself. One of the most popular and effective methods of doing so is through audio watermarking.

Audio Watermarking is a form of audio steganography that involves embedding information into an audio signal. The embedded information, called a watermark, usually contains metadata like the author's name, copyright information or a unique identifier that proves ownership. Audio replay devices or software can then detect such data and identify pirated copies. It is important that audio watermarking algorithms insert watermarks that are difficult to remove or alter without sacrificing the audio quality.

1.3 Design Requirements

When measuring the performance and efficacy of an audio watermarking algorithm, there are several important criteria to consider. These are shown in Figure 1.1 and described below as they are presented in this article [15]:

- **Imperceptibility**: the watermark must not be noticeable by the human ear. The importance of this criterion is twofold: first, an audible watermark might degrade the listening experience; second, an imperceptible watermark is less likely to be detected and removed by a malicious user.
- **Robustness**: the watermark must be resilient against common audio distortions and be able to withstand weaker audio processing attacks that are designed to not significantly damage the audio quality.
- **Security**: secure watermarking entails using encryption methods to ensure confidentiality. Authorized people are given an extraction key that is needed to retrieve the watermark from the watermarked audio.

- **Capacity**: capacity is measured by how many watermark bits are embedded in a second of the audio. a lower capacity algorithm requires longer audio files to be able to covertly insert longer messages.
- **Computational Complexity**: watermark algorithms should have reasonable computational complexity so that embedding and extraction can be performed efficiently.

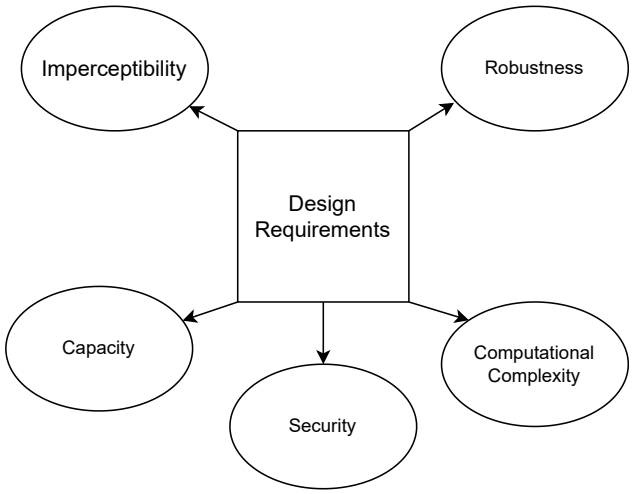

Figure 1.1: Design requirements for audio watermarking algorithms.

When evaluating a watermarking algorithm, it is important to determine how well it balances these criteria, as some of these requirements are often in conflict with one another. For example, a highly robust watermark may require more significant modifications to the audio signal that cause audible distortions, lowering imperceptibility. A well designed algorithm should be robust against common attacks, while maintaining good audio quality and a reasonable capacity and computational complexity. Our study focuses especially on the imperceptibility and robustness of the various analysed methods.

1.4 Types of Watermarking Methods

Audio Watermarking is not a new field of study. Many different methods have been proposed and researched over several decades. These can be categorised by the domain in which the watermark is embedded: **Time domain** and **Transform domain**. Figure 1.2 displays categorization for some of the most popular algorithms. Time-Domain watermarking is done by directly modifying the discrete audio samples, while Transform-Domain watermarking involves using transformations to convert the audio signal into a different representation before embedding the watermark there. Figure 1.3 represents a generic audio watermarking system.

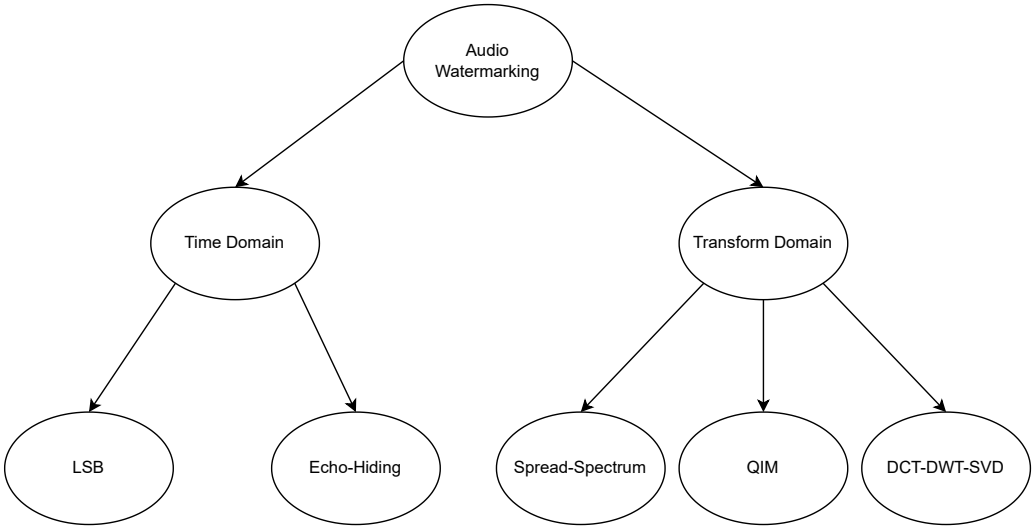

Figure 1.2: Types of audio watermarking algorithms.

Let *x(n)* note the original host signal in time domain. A typical model for embedding watermark *w(n)* that produces the watermarked signal *y(n)* is given by:

$$y(n) = x(n) + \alpha w(n) \tag{1}$$

where α represents the strength of the watermark. A higher α means higher robustness, but also lower imperceptibility. In equation (1) the watermark is added additively, but some methods embed multiplicatively. In these cases equation (1)

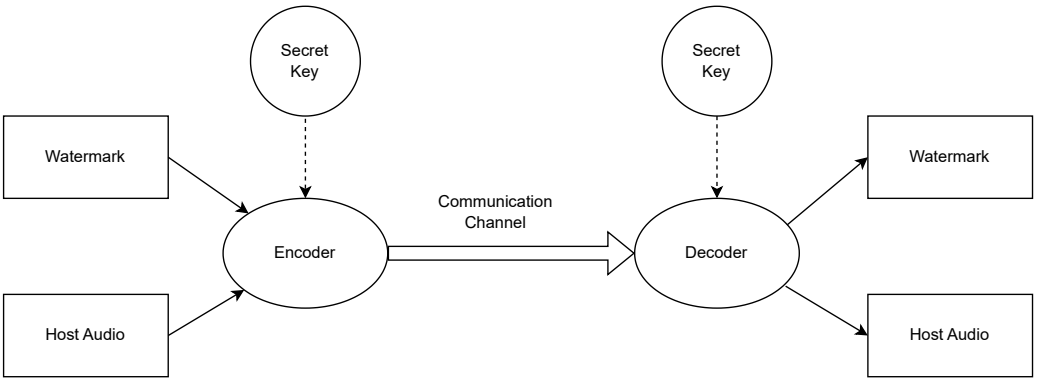

Figure 1.3: Generic audio watermarking system.

can be written as:

$$y(n) = x(n) \cdot (1 + \alpha w(n))$$

In transform domain, equation (1) becomes:

$$Y(k) = X(k) + \alpha W(k)$$

where $X(k)$, $W(k)$ and $Y(k)$ are the transformed representations of the original signal, watermark and watermarked signal respectively.

Figures 1.1, 1.2 and 1.3 were adapted from [15]

Chapter 2

Implementation of Algorithms

2.1 Audio Watermarking Algorithms

In the previous section we discussed two different types of audio watermarking algorithms divided into domain categories. Below we will discuss and explain some of the most popular algorithms from each category that we have implemented and tested.

Every implemented algorithm handles both mono and stereo audio. In the case of stereo files, each channel is watermarked similarly for the sake of redundancy so that the corrupted parts from the left channel may be recovered from the right channel and vice versa. The message to be embedded is in binary form with both text and audio formats supported. Every algorithm has at least one embedding and one extraction function. Every algorithm includes a blind extraction method, meaning it does not need the original audio signal to extract the watermark from the modified version.

2.1.1 Time Domain Methods

These methods embed hidden information by directly modifying audio samples. Their decoders detect and extract watermarks by examining how the samples were altered.

Least Significant Bit (LSB) Substitution

This is one of the simplest and most intuitive watermarking algorithms. The method relies on the fact that changing the least significant bits of the audio samples has minimal impact on audio quality. The algorithm works by replacing the least significant bits of each consecutive sample with bits from the secret message until the entire message has been embedded. Our implementation uses majority voting when modifying more than one LSB per sample.

While this algorithm is very simple and has high capacity, its robustness is poor. Small distortions can easily alter the least significant bits, muddying or even destroying the watermark. For this reason, LSB substitution serves as a good baseline for comparison but is not suitable for real-world applications. For more information on this method, refer to section 3.2.1 of [8].

Echo Hiding

This algorithm adds secret messages in the form of echoes of the host audio signal. Information is encoded by modifying delay Δt between the original and the echo. We use delays of Δt and $\Delta t'$ to represent binary '0' and '1' respectively, with both delays being small enough to be imperceptible to the human ear. The basic echo hiding scheme can only embed one bit per echo and as such, instead of using it on the entire audio signal, we must divide it into frames and embed one bit per frame. The base formula for embedding an echo is given by:

$$sw(n) = x(n) + \alpha x(n - \Delta t)$$

where α is the echo amplitude.

We implemented an encoder that uses a decaying kernel which adds multiple echoes with decreasing amplitude for improved robustness. The formula for the general echo-kernel is:

$$h(n) = \delta(n) + \sum_i \left[\underbrace{\overbrace{\alpha_{1,i}\delta(n - d_{1,i})}^{\text{Positive}} - \overbrace{\alpha_{2,i}\delta(n - d_{2,i})}^{\text{Negative}}}_{\text{Forward}} + \underbrace{\alpha_{1,i}\delta(n + d_{1,i}) - \alpha_{2,i}\delta(n + d_{2,i})}_{\text{Backward}} \right]$$

as described in section 3.1.1 of [17].

Our encoding function only uses the positive forward echoes for the sake of simplicity.

We first implemented a non-blind extractor that correlates each watermarked frame with the original for both possible delays and decides on the bit whose corresponding delay has a higher correlation. To get consistently good results we also had to subtract the host signal's autocorrelation.

For a blind extractor we decided on using cepstrum autocorrelation. The **cepstrum** is the result of computing the Inverse Fourier Transform of the logarithm of the estimated signal spectrum as shown in Figure 2.1 which was adapted from [7]. Its main use is investigating periodic structures in frequency spectra, produced in this case by echoes. The cepstrum of a signal containing an echo has an additive periodic component and thus exhibits a peak at each echo offset. In order to get better defined spikes, we applied a window to the watermarked signal before applying the Fast Fourier Transform (FFT), which reduces spectral leakage. The extractor compares peaks at both delays Δt and $\Delta t'$. Not mentioned in the patent we used as reference is that the cepstrum autocorrelation is the Inverse Fourier Transform (IFT) of the power spectrum as stated by the Wiener-Khinchin theorem.

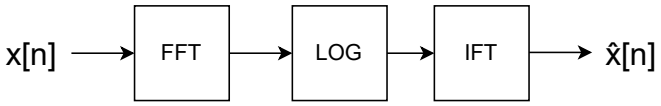

Figure 2.1: Cepstrum computation.

2.1.2 Transform Domain Methods

These methods utilize transformations to change the domain in which the audio signal is represented. The secret message is embedded in this alternate representation before the now watermarked audio is transformed back to the time domain.

In this project we use two popular and important transformations:

- **Discrete Cosine Transform (DCT)**: compared to the Fourier Transform (DFT), DCT only uses cosine functions and as such only yields real-valued

coefficients. This makes it computationally more efficient and better suited for audio processing.

- **Discrete Wavelet Transform (DWT)**: unlike DCT and DFT, which use sinusoidal basis functions, DWT uses wavelets that are localized in both time and frequency. This allows for improved imperceptibility by focusing on perceptually important components of the audio.

Spread Spectrum

Spread Spectrum is known for its inherent resilience against various signal processing attacks. The main idea behind this algorithm is to spread the hidden information across a broad range of frequencies. The host audio is once again split into frames. Each bit is embedded into the DCT domain. For this, we choose to encode in mid frequencies, as low frequencies more perceptible to the human ear and high frequencies are vulnerable to compression. We also require a pseudorandom (PN) sequence of length $k2 - k1$, where $k1$ - $k2$ is the range of bins in which information is embedded. Consider a single frame $X(k)$ and watermark b together with the spreading sequence $p(k)$. The embedding formula is given by:

$$Y(k) = X(k) + \alpha bp(k), \quad k_1 \leq k \leq k_2$$

where b is 1 if the embedded bit is '1' or -1 otherwise, α is the embedding strength and $X(k)$ is the DCT of the k-th frame. Decoding is done by simply computing the correlation of each frame in the watermarked signal and the same PN sequence used for encoding.

$$C =< Y, p >=< X, p > +\alpha b < p, p >$$

We choose a pseudo random sequence because it is statistically orthogonal to the host signal and therefore <X,P> ≈ 0. This means C $\approx \alpha$b<P,P> and because we chose $b \in \{-1, 1\}$, C will either be a negative or positive number. This allows us to correctly detect the embedded bit, as <P,P> has the same value regardless of the analysed frame from the watermarked signal. Ideally, <X,P> = 0 but this usually won't be the case. The expanded spread spectrum improves on this by

directly subtracting the host signal interference, giving the following embedding formula:

$$Y(k) = X(k) + (\alpha b - \lambda\Psi)p(k)$$

where $\Psi = \langle X, P \rangle$ and λ is a scaling factor between 0 and 1 that controls how much of the host signal interference is cancelled. The decoder stays unchanged. The main benefits of this improvement is that it makes detection more reliable and allows for choices of smaller alpha (since Var(<X,P>) lowers) which leads to more imperceptible watermarked audio. These explanations and formulas were adapted from section 3.1.2 of [17]. For further reading about improving the spread spectrum technique, refer to [9].

Quantization Index Modulation (QIM)

This method entails attributing an unique quantizer to each symbol of the message we wish to embed. For example, a binary message would require only two quantizers. These quantizers are used to alter the values of the DCT coefficients of the signal such that a decoder can tell which one was used over a set of consecutive values in order to correctly estimate the corresponding message symbol. By aiming for middle frequencies and changing coefficients by small amounts, we can ensure good imperceptibility.

Our QIM encoder groups audio samples into frames of the same size and enumerates through the symbols of the watermark in bits form. For each bit in the message, we take the DCT coefficients of a frame and snap them to the nearest value of the bit's specific quantizer. Since our watermark is in binary, we have Q_0 for bit 0 and Q_1 for bit 1 with the quantizers being offset versions of each other (For step size = 10, we have Q_0 = 0, 10, 20, 30, … and Q_1 = 5, 15, 25, …). We also utilize frame overlapping in order to prevent abrupt changes between frames when we have alternating bit values. The decoder goes through the same frames and estimates which quantizer was likely used to decide the current bit's value. Our approach is based on the one described in [3].

In addition to the basic QIM algorithm, we have found and implemented two improved versions:

- **Dither**: QIM-Dither rectifies a problem produced by the way information

is embedded. The QIM algorithm may produce periodic distortion patterns in the watermarked signal, which can be noticed by attackers. To prevent this, a small random value called a dither is added to each coefficient before quantization. This randomizes the distortion patterns and makes it more difficult for an attacker to identify and remove the watermark. For a more in-depth explanation, refer to section 3.1.3 of [17].

- **DWT**: This version of QIM uses the DWT instead of the DCT when transforming the host signal. This allows for better imperceptibility and robustness by taking advantage of the time-frequency localization properties of wavelets.

DCT-DWT-SVD Algorithm

This algorithm was proposed in [5] and combines the DCT and DWT transformations with Singular Value Decomposition (SVD) to achieve a robust and imperceptible watermarking scheme. The previously mentioned benefits of DCT and DWT are leveraged alongside many useful properties of SVD such as stability, scalibility and proportionality. This algorithm embeds watermark data into the highest singular values of DWT-DCT transformed audio frames. This ensures a higher robustness as singular values are much less affected by common signal processing attacks.

While this initial version only includes a non-blind extractor, we have modified the algorithm to allow for blind extraction by using the predefined quantization coefficient Q in a similar manner to QIM. The additional predefined constants K1 and K2 are not used in our implementation.

2.2 Reed Solomon Codes

Error Correction is the process of detecting and fixing errors that appear when transmitting data via communication channels. Forward Error Correction is a technique for controlling such errors by sending additional, redundant data alongside the original message. This added information is used to recover any parts that are lost or corrupted from the original message after the transmission.

Reed Solomon (RS) codes are a type of Forward Error Correction codes that are effective for correcting burst errors. In the context of audio watermarking, RS codes can be used to reinforce the robustness of the watermark and make them more recoverable after a signal processing attack.

The math behind these codes is based in the arithmetic of finite fields (Galois Fields or GF for short). We have used an implementation over $GF(2^8)$, meaning each byte of the watermark signal is treated as a symbol of the alphabet. Before embedding the watermark into the host signal, the bytes of the watermark message are first turned into a 255 byte long codeword. At extraction, the 255 byte codeword is extracted and then decoded into the original watermark. In the case that the watermark is too long, then multiple codewords will have to be embedded into the host signal and later extracted. After extraction, each codeword will be decoded and the results will be concatenated to reproduce the original watermark.

The encoding method we have chosen is the polynomial evaluation method, whereby the message symbols (in our case, the individual bytes of the watermark) are elements of the finite field $GF(256)$ and are interpreted as the coefficients of a message polynomial of degree at most $k-1$, where k is the number of message symbols.

A fundamental property exploited by Reed-Solomon codes is that a polynomial of degree at most $k-1$ is uniquely determined by its values at k distinct points. Using this property, the message polynomial is evaluated at n distinct field elements - specifically, consecutive powers of a primitive element α of $GF(256)$ - to produce an n-symbol codeword. The more common implementation we have found is the systematic encoding, but we found it more difficult to understand so we did not use it. As a consequence of using a non-systematic encoding, the message symbols are not directly present in the codeword.

Because we did not use the systematic encoding approach, we were forced to embed the entire 255-byte codeword, and also perform a Lagrange interpolation after decoding in order to obtain the original message polynomial. More thorough explanations can be found in [10] and [13].

The decoder uses the general BCH code decoding strategy [1].

Our implementation was inspired by the ones from [11] and [6].

Chapter 3

Experiments and Results

3.1 Setup

We conducted experiments on a dataset of 20 audio excerpts of equal duration from different Minecraft songs. First we show the robustness vs. attack strength graphs, as described on page 113 of [8]. They show the bit error as a function of the attack strength for a fixed audio quality. For each algorithm and attack strength, watermark extraction was performed on all test signals, and the resulting bit error rates were averaged. Embedding parameters were selected such that all watermarking algorithms produced comparable perceptual audio quality, as measured by Objective Difference Grade (ODG). This metric's values range from 0 (imperceptible difference) to 4 (very annoying distortion). More details about it can be found in section III of [15]. We aimed for algorithm parameters that would yield an average ODG between -0.5 and 0.5 across all test audio files. Below is a table containing the selected parameters for each algorithm:

Each algorithm was subjected to a different suite of attacks. We also showcased how Reed Solomon codes impacted the robustness of the watermarking schemes against these attacks. All evaluated watermarking schemes perform blind extraction but are not synchronization-invariant. Therefore, attacks involving time-domain editing, such as cropping or segment replacement were excluded as they would disrupt watermark synchronization and may lead to complete decoding failure.

Algorithm	**Parameters**
LSB	`num_lsbs = 6`
Echo Hiding	`alpha = 0.035, d0 = 50, d1 = 100, decay = 0.2, length = 1`
Spread Spectrum	`alpha = 4.5, k1 = 100, k2 = 400`
QIM	`delta = 150, overlap = 512`
DWT-QIM	`delta = 75, overlap = 512, wavelet = 'db4', level = 4, embed_band = 3`
DCT-DWT-SVD	`step_size = 200`

Table 3.1: Selected embedding parameters for each watermarking algorithm.

For normal testing we used 30 second long audio signals and for Reed Solomon encoded signals we used longer 90 second audio signals as it required a 255-byte long payload (for an entire codeword).

We still used a form of cropping attack that replaces a contiguous segment of the watermarked audio with the corresponding segment from the original (unwatermarked) signal, as is utilised in [5].

For the mp3 compression attack, we used the 90 second files for testing. This is because the mp3 algorithm used disproportionately affects shorter files.

We also used a subjective testing measure known as the AXB test described in section III A of [15]. The subject is presented with the audio signals A and B (the original and watermarked audio signals in random order) and the audio signal X (the original audio signal), and is tasked with guessing which between A and B is most similar to X. We asked 4 university students to partake in this test and the results are shown in Table 3.2.:

3.2 Results

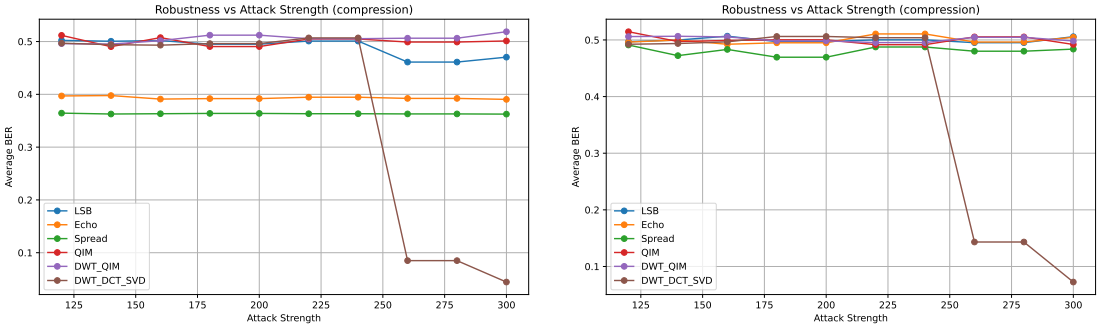

Figure 3.1: Robustness vs. attack strength under MP3 compression (left) and with Reed-Solomon coding (right).

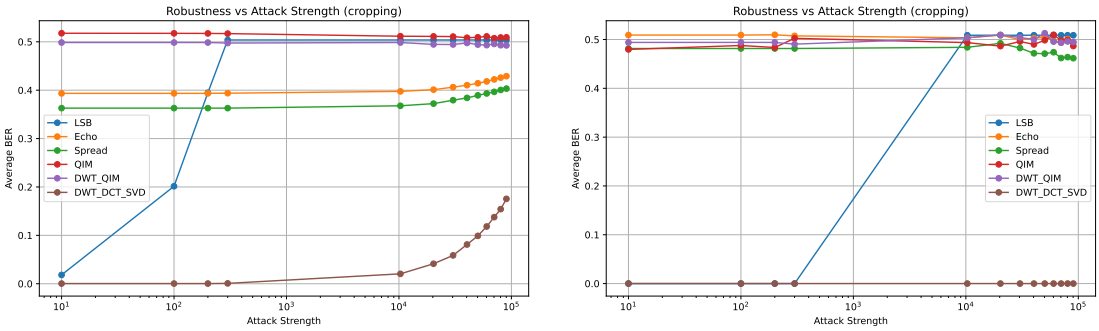

Figure 3.2: Robustness vs. attack strength under cropping attack (left) and with Reed-Solomon coding (right).

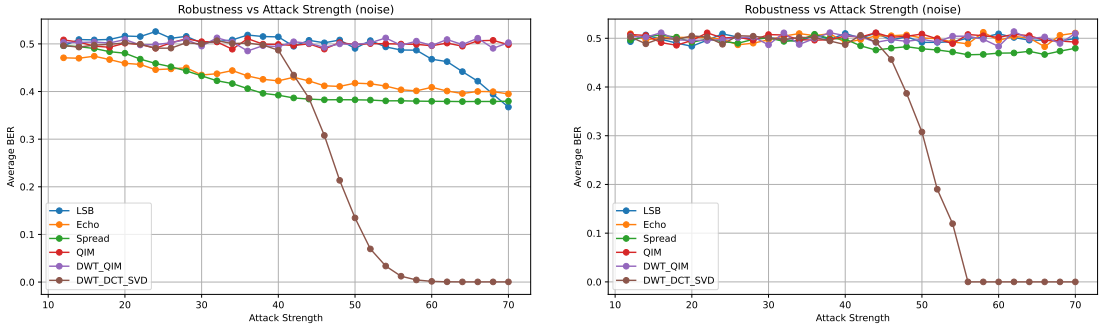

Figure 3.3: Robustness vs. attack strength under noise addition (left) and with Reed-Solomon coding (right).

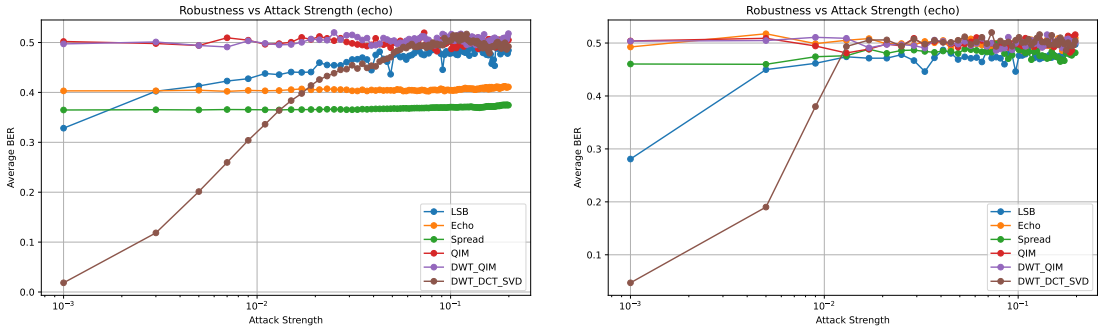

Figure 3.4: Robustness vs. attack strength under echo attack (left) and with Reed-Solomon coding (right).

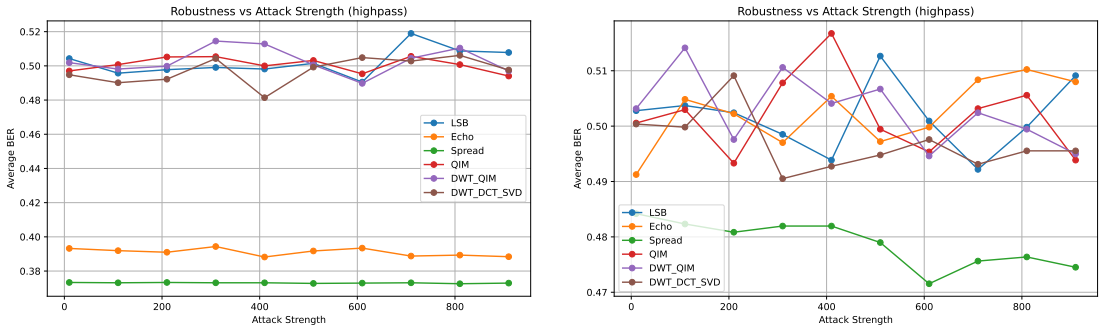

Figure 3.5: Robustness vs. attack strength under highpass filtering (left) and with Reed-Solomon coding (right).

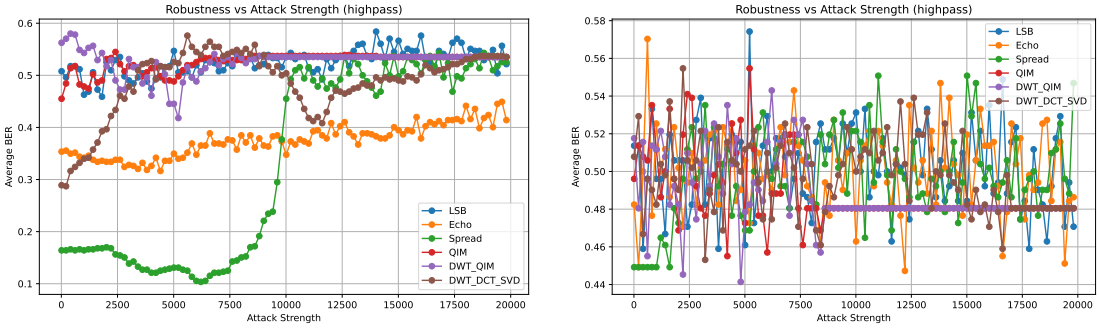

Figure 3.6: Robustness vs. attack strength under highpass filtering (left) and with Reed-Solomon coding (right) with hand-picked audio files.

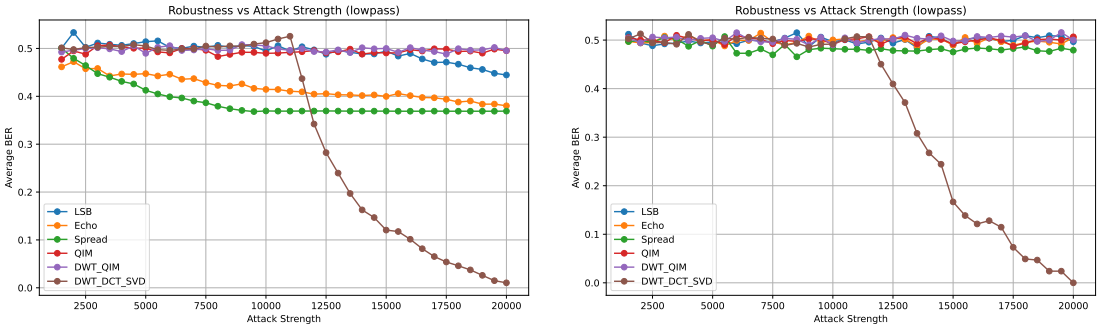

Figure 3.7: Robustness vs. attack strength under lowpass filtering (left) and with Reed-Solomon coding (right).

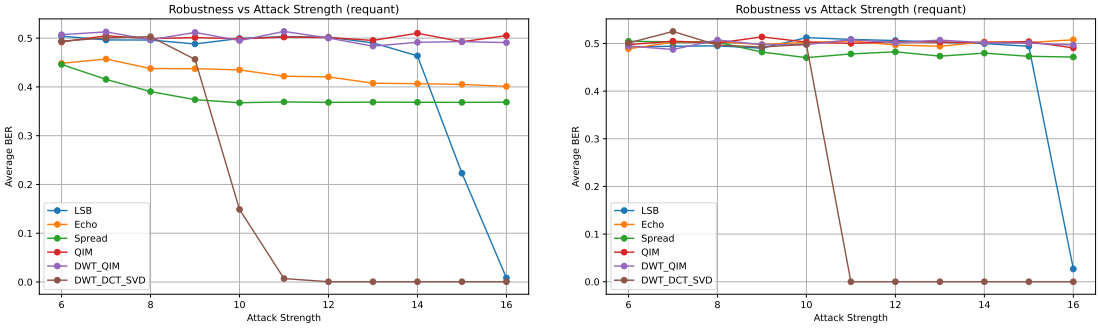

Figure 3.8: Robustness vs. attack strength under requantization (left) and with Reed-Solomon coding (right).

Algorithm	Guessed Correctly (%)
LSB	45.24
Echo Hiding	38.10
Spread Spectrum	30.95
QIM	75.00
DWT-QIM	38.10
DWT-DCT-SVD	61.90

Table 3.2: AXB test average correctness per algorithm.

Chapter 4

Conclusions

From our results highlighted in the previous chapter we have drawn the following conclusions:

- The DCT-DWT-SVD algorithm performed the best across the tests. This performance highlights the benefit of combining multiple embedding stages, where each transform contributes complementary strengths. This reduced the vulnerability of the method to any single class of attack, resulting in improved overall robustness compared to standalone watermarking techniques. Following that, Echo Hiding and Spread Spectrum performed generally fine, while LSB and QIM variants are the weakest.

- Reed Solomon codes showed significant improvement in the case of the crop-replace attack, for the algorithms that were not totally decimated by it, showcasing its ability of correcting burst errors and its inability of correcting too many errors. For attacks such as additive noise, RS codes actually decreased the performance of all algorithms, which is explained by the fact that errors were spread across many watermark symbols.

- In the high-pass filtering scenario, we observed that audio samples with a more “jumpy” structure — i.e., less linear or slowly varying baseline content — tended to yield improved BER across all algorithms. We decided to pick the files ”5.wav” and ”11.wav” to be tested on separately, as is shown in Figure 3.6. In this case, Spread Spectrum performed surprisingly well,

which could be attributed to the fact that the watermark energy is spread across many midband frequencies in the DCT domain.

- The AXB results show that all algorithms, except default QIM were imperceptible to the listeners (with the parameters mentioned above).

Chapter 5

Possible Improvements and Future Work

We have identified several areas that could use improvement such as:

- Given that both QIM and DWT-QIM performed surprisingly bad, the way they are implemented could have buggy or inefficient code.
- We have used our own implementation of RS codes; validating results against a well-established reference implementation could provide clearer insight into its actual effectiveness.
- ODG scores were computed using the Aqua-tk Python library; while convenient, this implementation may not fully replicate the reference ITU-R BS.1387 (PEAQ) standard and could introduce inaccuracies.
- The AXB test involved only a few participants, increasing the number of test subjects would be required for a more statistically robust conclusion.

Some ideas for future work include:

- The test suite could be improved upon, broader and more diverse set of attacks and testing conditions.
- Synchronization-invariant watermarking schemes could be implemented and evaluated to better address time-scale and cropping attacks.

- This study focused solely on pure DSP algorithms. New state-of-the-art algorithms utilize sophisticated machine learning techniques. Researching these methods is required for getting a full picture of the audio watermarking field.

Bibliography

[1] *BCH code*. URL: https://en.wikipedia.org/wiki/BCH_code.

[2] Steve Brunton. *Wavelet and Multiresolution Analysis*. June 2020. URL: https://www.youtube.com/watch?v=y7KLbd7n75g&list=PLMrJAkhIeNNT_Xh3Oy0Y4LTj0Oxo8GqsC&index=33.

[3] Brian Chen and Gregory W. Wornell. "Quantization Index Modulation: A Class of Provably Good Methods for Digital Watermarking and Information Embedding." In: *IEEE Transactions on Information Theory* 47.4 (May 2001), pp. 1423–1443. DOI: 10.1109/18.920252. URL: https://sia.mit.edu/wp-content/uploads/2015/04/2001-chen-wornell-it.pdf.

[4] Silicon DSP Corporation. *OFDM Tutorial Series: Reed Solomon Coding*. June 2021. URL: https://www.youtube.com/watch?v=K26Ssr8H3ec.

[5] Pranab Kumar Dhar and Tetsuya Shimamura. "A DWT-DCT-Based Audio Watermarking Method Using Singular Value Decomposition and Quantization." In: *Journal of Signal Processing* 17.3 (May 2013), pp. 69–79. URL: https://scispace.com/pdf/a-dwt-dct-based-audio-watermarking-method-using-singular-312a621fud.pdf.

[6] Tomer Filiba et al. *Reed-Solomon Error Correction Library*. https://github.com/tomerfiliba-org/reedsolomon.

[7] Daniel Gruhl, Anthony Lu, and Walter Bender. "Echo hiding." In: *Information Hiding*. Ed. by Ross Anderson. Berlin, Heidelberg: Springer Berlin Heidelberg, 1996, pp. 295–315. ISBN: 978-3-540-49589-5. URL: https://www.freepatentsonline.com/5893067.pdf.

[8] Stefan Katzenbeisser and Fabien A. P. Petitcolas. *Information Hiding Techniques for Steganography and Digital Watermarking*. Boston, MA: Artech House, 2000. ISBN: 1580530354.

[9] Henrique S. Malvar and Dinei A. F. Florêncio. "Improved Spread Spectrum: A New Modulation Technique for Robust Watermarking." In: *IEEE Transactions on Signal Processing* (). URL: https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/MalvarFlorencioTSPApr03.pdf.

[10] *Reed-Solomon Codes*. URL: https://courses.cs.duke.edu/spring10/cps296.3/rs_scribe.pdf.

[11] *Reed-Solomon codes for coders*. Wikiversity. URL: https://en.wikiversity.org/wiki/Reed-Solomon_codes_for_coders.

[12] *Reed-Solomon error correction*. URL: https://en.wikipedia.org/wiki/Reed%E2%80%93Solomon_error_correction.

[13] Reed I. S. and Solomon G. "Polynomial Codes Over Certain Finite Fields." In: *Journal of the Society for Industrial and Applied Mathematics* 8.2 (June 1960), pp. 300–304. URL: https://sites.math.rutgers.edu/~zeilberg/akherim/reed.pdf.

[14] Simcenter Testing. *Cepstrum Analysis*. Nov. 2024. URL: https://www.youtube.com/watch?v=TmNo18IFqkM.

[15] Mohammad Shorif Uddin, Ohidujjaman, Mahmudul Hasan, and Tetsuya Shimamura. "Audio Watermarking: A Comprehensive Review." In: *International Journal of Advanced Computer Science and Applications (IJACSA)* 15.5 (2024), pp. 1410–1418. URL: https://thesai.org/Downloads/Volume15No5/Paper_141-Audio_Watermarking_A_Comprehensive_Review.pdf.

[16] vcubingx. *What are Reed-Solomon Codes? How computers recover lost data*. Apr. 2022. URL: https://www.youtube.com/watch?v=1pQJkt7-R4Q.

[17] Yong Xiang, Guang Hua, and Bin Yan. *Digital Audio Watermarking Fundamentals, Techniques and Challenges*. SpringerBriefs in Signal Processing. Singapore: Springer, 2017. ISBN: 978-981-10-4288-1. DOI: 10.1007/978-981-10-4289-8. URL: http://ndl.ethernet.edu.et/bitstream/123456789/36072/1/32.pdf.